

[WinCVS](#) › [FrontPage](#) › [개발자코너](#) › CVS/GuideLine
Guidelines for the use of CVS

Login:
Password:
[Join](#)

- 작성자: Tony Finch <dot at dotat.at> \$dotat: doc/web/writing/cvs-guidelines.html,v 1.9 2002/12/17 14:14:58 fanf2 Exp \$
원문: <http://dotat.at/writing/cvs-guidelines.html>
- 번역: 최정대 <whitekid at netian.com>
번역원문: <http://wiki.comdongin.com/CvsGuideLine>

Don't be overly suspicious where it's not warranted.

[Adv-Routing-HOWTO](#)
[LinuxdocSgml/BogoMips](#)
[xbitmap](#)
[JbpmConfiguration](#)

Contents ▲

- [1 소개](#)
- [2 Basic points](#)
 - [2.1 check in 해야 하는 때](#)
 - [2.2 Commit 메시지](#)
 - [2.3 tag 사용하기](#)
 - [2.4 The modules 파일](#)
- [3 코드](#)
 - [3.1 절대로 코드 형식을 변경하지 마라](#)
 - [3.2 코드 포맷을 일관되게](#)
 - [3.3 탭 설정](#)
 - [3.4 주석](#)
 - [3.5 CVS ident 문자열](#)
- [4 문서](#)
 - [4.1 문서를 재포맷하지 마세요](#)
 - [4.2 CVS ident 문자열](#)
- [5 Importing code](#)
 - [5.1 Importing local code](#)
 - [5.2 업스트림 코드를 импорт하기](#)
 - [5.3 업스트림 코드를 업데이트하기](#)
- [6 애매한 상황 처리하기](#)
 - [6.1 디렉터리 만들기](#)
 - [6.2 "이런! 잘못된 것을 check in 해버렸네..!"](#)
 - [6.3 "Whoops! I cocked up a cvs import!"](#)
 - [6.4 파일을 바꾸기](#)
 - [6.5 지운파일 복구하기](#)
 - [6.6 사용자 정의 Tag](#)



1 소개 ¶

[[edit](#)]

이 문서는 CVS를 사용하는 약간의 가이드라인에 대해서 설명하고 있으며, Demon Internet에서 사용하기 위하여 작성하였고, 또한 많은 곳에서 유용할 것으로 믿습니다. 이 문서는 독자는 이미 CVS에 대해서 기본 지식을 가지고 있는 것으로 가정합니다. 튜토리얼이 필요하다면 Pascal Molli's CVS page와 특별히 Per Cederqvist's manual를 참조하시기 바랍니다.

CVS가 효과적으로 사용되어질 때는 개발자에게는 CVS가 저장소(repository)에 파일의 변경사항 기록을 저장하고 있기 때문에 값어치 있는 툴입니다. 그러므로 이 가이드라인의 많은 목적은 최초 개발자와 나중에 코드를 관리하는 사람간에 변경기록을 가능한 한 유용하게 만들기 위함에 목적을 두고 있습니다.

CVS는 잘못을 용서하지 않는다: 저장소에 변경이 한번 가해지면 일반적으로 되돌릴 수 없습니다. 그러므로 이 가이드라인의 약간 부분은 경험 많은 CVS 사용자들이 이러한 부분을 수정하기 위하여 어떤 다양한 작업을 수행하는지에 대해 설명할 것입니다.

2 Basic points ¶

[[edit](#)]

이 섹션에서는 CVS를 사용하는데 일반적으로 적용할 수 있는 사항을 설명합니다; 가이드라인의 다른 섹션에서는 특별한 각각의 작업 또는 사용법에 대해서 좀더 자세하게 제시할 것입니다.

2.1 check in 해야 하는 때 ¶

[[edit](#)]

Check in은 빨리, check in은 자주. 작업중에 수정을 가했을 경우 check in하라. 변경이 생길 때마다 개별 커밋으로 check in 하라(가능한 한). 작업중에 최소한의 기능이 추가되거나 최소한 컴파일 오류없이 컴파일되면 check in하는 것을 주저하지 말아라.

2.2 Commit 메시지 ¶

[\[edit\]](#)

커밋 메시지를 의미있게 작성하라. 이 커밋이 어떤 버그를 수정하는지 또는 어떤 기능이 추가되었는지를 설명하라. 의미를 압축하지 말라: "fixed typo"라는 메시지는 너무 짧다. "fixed typo in error message" 또는 "fixed typo in function name"은 좋다. 이렇게 함으로써 커밋 메시지만으로 원하는 변경사항을 쉽게 찾을 수 있게 된다(cvsweb에 의해 표시되는 메시지 참조).

이러한 방식은 많은 정보를 포함한다. CVS는 자동적으로 커밋하는 날짜와 시간, 누가 커밋을 했는지, 코드가 어떻게 바뀌었는지 등등을 관리한다. 커밋 메시지에 이러한 사항들을 포함할 필요 없다.

2.3 tag 사용하기 ¶

[\[edit\]](#)

미심적인 부분이 있다면 tag를 작성하라. Tag는 코드의 특별한 버전을 유지하는데 유용하다. 예를 들면 서비스를 수행하고 있는 중이거나, 또는 커다란 변화를 가하거나 import 하기 바로 직전의 경우다. Tag는 또한 독립적인 branch로 사용된다. 예를 들면 webmail-19990811, pre-new-resolver, fanf-patches는 위에서 언급한 용법을 보이고 있다. Tag는 modules file에 기록된다.

2.4 The modules 파일 ¶

[\[edit\]](#)

modules 파일에 대한 코멘트를 작성해라. 이것은 저장소에 있는 모듈을 정의한다. 가장 간단한 경우는 저장소 디렉터리의 단순 별명이다. CVS는 또한 여러 디렉터리를 결합해서 모듈의 형태로 모을 수 있다. 파일에 있는 각 모듈에 대해서 모듈의 내용과 언제, 누구에 의해서 만들어졌는지, 그리고 모듈에 사용되어진 Tag와 branch에 대한 설명을 작성할 수 있다.(Tag는 커밋 branch에 반해서 메시지를 가질 수 없다.)

3 코드 ¶

[\[edit\]](#)

이 섹션의 대부분은 일반적인 사항이지만, CVS에서 반복되는 작업을 하는 동안에는 명확하게 의의가 다가오지 않기 때문에 유용할 것이다.

3.1 절대로 코드 형식을 변경하지 마라 ¶

[\[edit\]](#)

절대로 코드 형식을 변경하지 마라. 이것은 diff를 이해하기 어렵고, 적용하기 어렵게 만들기 때문에 진짜 위험한 일이다. 이전 버전의 작성자는 재포맷된 코드의 패치를 받아들이지 않을 것이다. 상위 버전에 대한 버그 수정과 패치는 적용되지 않을 것이다. 상위 버전 코드의 수정된 버전은 받을 수 없을 것이다. 실제 변경사항은 재포맷된 많은 정보에 가려 찾기 힘들 것이다.

어떤 사람의 좋아하는 코딩 스타일은 다른 사람의 것보다 아주 특별하게 좋거나 나쁘지가 않다. 그러므로 코드의 재포맷은 얻는 불이익에 비해서 얻는 이익이 없다.

3.2 코드 포맷을 일관되게 ¶

[\[edit\]](#)

코딩할 때는 같은 코딩스타일을 사용하라. 이것은 이전 서브섹션의 직접적인 결과이다. 코드를 읽는 사람에게는 일관된 룰에 의해 작성된 코드가 쉽게 읽힌다. 따라서 다른 사람이 작성한 코드에 새로운 코드를 추가할 때는 같은 스타일로 추가하라.

3.3 탭 설정 ¶

[\[edit\]](#)

탭은 8자로 설정한다. 이것도 또한 이전의 항목과 연관되어있다. 들여쓰기 크기는 다양하게 설정될 수 있기 때문에(일반적으로 8글자이다) 설정을 변경하여 쓰는 것은 혼란과 재포맷을 여지가 있다. 탭 크기를 4로 설정하는 것이 당신의 스타일에 맞지만 나머지는 코드가 엉망이라고 생각할 것이다. 탭크기 4를 원하면 vim에서는 sts=4:sw=4:ts=8로 한다.

3.4 주석 ¶

[\[edit\]](#)

커밋 메시지는 주석에 적합하지 않다, 반대의 경우도 마찬가지다. 주석은 데이터 구조, 왜 코딩을 그렇게 했는지 그리고 코드가 무엇을 하는지에 대해서 설명해야 한다. 커밋 메시지는 왜 코드가 수정되었는지에 대한 메시지를 포함해야 한다.

3.5 CVS ident 문자열 ¶

[\[edit\]](#)

CVS \$Header: /home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine,v 1.14 2006/07/20 01:18:07 kss Exp kss \$ 문자열을 코드에 포함하여라. 이것은 현재 파일이 어떤 버전이고 어디서 왔는지에 대한 정보를 알기 쉽게한다. 이렇게 하면 파일의 버그와 수정사항을 CVS history에서 찾아내는데 유용하다. 예를 들면 C 소스 코드에서는 파일의 상위부분에 static const char *const cvsid = "\$Header: /home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine,v 1.14 2006/07/20 01:18:07 kss Exp kss \$";을 포함한다. 이렇게 하면 버전 정보가 컴파일된 바이너리에 포함된다. 그리고 C 헤더파일과 스크립트에도 \$Header: /home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine,v 1.14 2006/07/20 01:18:07 kss Exp kss \$을 처음에 포함하라.

만일 저장소가 \$Header: /home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine,v 1.14 2006/07/20 01:18:07 kss Exp kss \$ 대신하는 적합한 사용자 정의 tag(custom tag)가 설정되어 있다면 사용자 정의 tag를 사용한다. 자세한 사항은 사용자 정의 Tag를 참조하시오.

4 문서 ¶

[\[edit\]](#)

이 섹션의 목적은 이전과 비슷합니다.

4.1 문서를 재포맷하지 마세요 ¶

[\[edit\]](#)

이 이유는 문단이 적당히 나누어졌다면(wrapped properly) 읽기 쉽기는 하지만 코드를 재포맷하면 안되는 이유와 같습니다. 그러므로 문단을 편집할 때 re-wrap하는 것은 좋은 생각이지만, 나머지 문서를 수정하는 것은 이유가 없습니다.(?)

4.2 CVS ident 문자열 ¶

[\[edit\]](#)

문서도 CVS \$Header: /home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine,v 1.14 2006/07/20 01:18:07 kss Exp kss \$ 나 적당한 사용자 정의 tag를 시작부분에 포함하여야 합니다.

5 Importing code ¶

[\[edit\]](#)

코드를 임포트하는 것은 꽤 쉽습니다. 그렇지만 부주의한 임포트는 수정하기 어려운 저장소의 혼란을 가져옵니다.

5.1 Importing local code ¶

[\[edit\]](#)

순서는 다음과 같습니다.:

1. 저장소 위치(\$loc)를 선택합니다. 이곳은 홈디렉터리 아래나 주어진 서비스나 함수나 함수가 가 관련 있는 디렉터리 입니다. 저장소를 깔끔하게 유지하십시오.
2. vendor tag(\$v)와 a release tag(\$)를 선택하세요. vender tag는 회사 이름이나 당신의 이름 정도면 되고 release tag는 "Start"나 "Initial"같은 것이면 되겠습니다.
3. 만일 작업중인 파일이 없는 새로운 프로젝트라면, 최초의 빈 디렉터리를 만드십시오. 안그런가요? If not, why didn't you import it earlier?
4. 프로젝트 디렉터리의 가장 상위 디렉터리에서 cvs import \$loc \$v \$r 를 실행하세요(해당 변수를 적당히 설정합니다.). 그런 다음 "initial import of my foo program which bars customers"와 같은 적당한 커밋 메시지를 입력하세요.
5. 원 프로젝트와 충돌하지 않는 상위 디렉터리로 이동하여 "cvs checkout" 명령으로 CVS에서 checkout을 하십시오. 모든게 정상적으로 진행된다면 두개의 동일한 프로젝트 복사본을 가

- 지게 됩니다. 이전의 프로젝트 디렉터리는 지우고 새로운 작업디렉터리가 만들어졌습니다.
6. 큰 프로젝트의 부분 작업이 아니라면 modules 파일에 새로운 프로젝트를 추가하십시오.

5.2 업스트림 코드를 임포트하기 ¶

[\[edit\]](#)

여기의 순서는 기본적으로 전에 설명했던 것과 동일하지만 다음 사항을 고려하십시오.

1. vendor tag는 vendor의 실제 이름이어야 합니다. 예를 들면 bind와 inn의 distributor 는 "ISC"
2. release tag는 소프트웨어의 이름과 버전 번호여야 합니다; 하이픈과 점은 밑줄(_)로 대체합니다. 예) "bind_8_2_1", "inn_2_2".
3. tags는 module 파일에 documented 되어야 합니다.
4. "cvs import" 명령은 소스가 있어야 하는 최상위 디렉터리에서 실행되어야 합니다. 때로는 소프트웨어는 분리된 tarball(예를 들면 소스와 문서로)로 배포됩니다. 그리고 새로운 디렉터리의 가장 최상위 디렉터리에 있어야 합니다.
5. 커밋 메시지는 소프트웨어가 어디에서 왔는지 언급해야 합니다. 예)
[`<ftp://ftp.isc.org/isc/bind/src/8.2.1>`](http://ftp.isc.org/isc/bind/src/8.2.1)

5.3 업스트림 코드를 업데이트하기 ¶

[\[edit\]](#)

비슷한 순서를 반복하지만, 처음과 마지막에 몇 가지 단계가 추가됩니다.

1. 상위 버전의 소스로 업데이트하기 전에, 변경된 버전을 tag하십시오; 작업 디렉터리에서 "cvs tag bind_8_2_1_local" 명령을 이용하여 이전 버전 번호를 이용하여. 다른 것으로는 before_bind_8_2_2와 같이 사용할 수 있습니다. 이러한 것은 나중에 현재 버전의 코드를 쉽게 얻을수 있게 만들어줍니다. modules 파일에 tag를 문서화시키십시오.
2. 이전 섹션의 방법으로 새 버전을 임포트 하십시오. tarball은 새 디렉터리 트리로 만들어 질 것입니다. vender tag는 이전과 같아야 하며, release tag는 새로운 버전 번호호를 반영해야 합니다. 그리고 커밋 메시지는 distribution site를 변경되지 않았으면 언급할 필요는 없습니다.(?)
3. 임포트 후에는 아마도 해결해야 할 conflict를 있을 것입니다; 대부분은 CVS가 자동적으로 해결해주지만, 사용자의 수정 때문에 수동으로 해야 하는 경우도 있습니다. CVS will tell you the command to run to resolve the conflicts; as before care should be taken to avoid mixing up the pristine upstream source, your old working directory, and the newly checked out source, by moving directories that may be overwritten out of the way.
4. CVS가 자동으로 해결할 수 있는 conflict를 해결한 후, 남아있는 나머지 conflict를 해결하십시오. 코드에서 "<<<<", "====", ">>>>"을 포함하고 있는 것으로 conflict를 찾아낼 수 있습니다. conflict를 해결한 후에는 수정된 코드를 check in 하십시오. 간단한 커밋 메시지는 "resolve import conflicts" 같은 것이 좋습니다.
5. 만일 첫 번째 단계에서 before_ 스타일의 tag를 쓴다면 여기서 post-import tag를 추가하고 싶을지도 모른다. 예)after_bind_8_2_2.

6 애매한 상황 처리하기 ¶

[\[edit\]](#)

CVS의 특정한 작업, 특히 실수를 복구하는 기능 CVS의 기능적 제한 때문에 복구하기 힘듭니다. 저장소를 직접 수정하는 것은 아주 안 좋은 아이디어이지만, 가끔은 피할 수 없다. 이 가이드라인에서는 이러한 상황을 어떻게 해야 하는지에 대해서 설명한다.

6.1 디렉터리 만들기 ¶

[\[edit\]](#)

cvs import 명령을 사용하여 새 최상위 디렉터리를 만듭니다. 저장소에 디렉터를 추가하려면 관련있는 섹션 5.1을 참조하십시오. 이미 있는 서브디렉터리를 추가하려면 작업 디렉터리에 디렉터리를 만들고 cvs add명령을 사용해 추가할 수 있다. 디렉터리는 바로 만들어질 것이며 cvs commit 명령을 사용할 필요가 없다.

6.2 "이런! 잘못된 것을 check in 해버렸네..!" ¶

[\[edit\]](#)

한 번 커밋된 변경은 다시 되돌릴 수 없습니다. 변화를 다시 되돌리고 이전 코드를 새 리비전으로

check in 해야합니다.

어떤 때는 작업 버전에 많은 따로따로 커밋 되어야하는 변화사항을 포함하고 있을 수 있습니다. 실수로 이러한 것들을 커밋할 때 모든 변화사항을 커밋 메시지로 기록하지 않고 그 중에 하나만의 변경을 기록하는 경우도 있습니다. 이러한 실수를 되돌리는 안전한 방법은 커밋된 것을 다시 되돌리고 올바른 메시지로 다시 커밋하는 것입니다; 직접적으로 레포지트리를 수정하는 것은 아주 위험한 일입니다.

6.3 "Whoops! I cocked up a cvs import!" ¶

[\[edit\]](#)

import 권한을 갖는다는 것은 중요하다. 왜냐 Import 권한은 장기간 사용되는 레포지트리에 영향을 미치기 때문이다. import 명령을 사용할 때는 특별히 주의 깊게 다시 한번 검토해보라!

만일 실수를 했다면, 해결 방법은 정확히 무엇이 잘못되었느냐에 따라서 결정된다. 명령을 잘못된 작업 디렉터리에서 사용하거나 잘못된 레포지트리 패스의 사용 등등일 것이다. 중요한 점은 임포트된 파일이 레포지트리의 파일과 같은 공간을 차지하는지 아닌지 여부이다.

만일 잘못 임포트된 파일중에 어떤 파일도 레포지트리에 이미 존재하고 있는 파일과 같은 이름을 갖지 않는다면, rm 명령을 이용하여 단순히 레포지트리에 지우는 것으로 수정할 수 있다. 만일 Import가 잘못된 tag로부터 분리되어 괜찮다면(OK), tag는 아마도 많은 수고 없이 삭제하고 다시 정확하게 적용할 수 있을 것이다.(이것은 잘못된 vendor branch tag의 경우는 적용되지 않을 것이다.) 만일 관련 없는 파일과 파일이름이 충돌되었다면, 그것은 완전히 심각한 문제이다. CVS Guru 찾아 레포지트리를 수동으로 고치도록 요청해라. 아마 환영 받지 못할 것이다.(^^)

6.4 파일이름 바꾸기 ¶

[\[edit\]](#)

레포지트리를 수동으로 수정해야 하는 필요가 있는 가장 빈번한 상황이 있는데, 그것은 파일을 옮기는 것이다. 목적은 파일을 새로운 위치에 완전한 히스토리를 유지하면서 이동하면서 여전이 이전의 checkout를 작동 가능하게 만드는 것이다. 순서는 다음과 같다.

CVS 서버로 로그인 하고 적당한 ".v"파일을 이전의 위치에서 새로운 위치로 복사한다. 작업 디렉터리에서 cvs update 명령을 사용한다. 그렇게 하면 이전 것과 새로운 것 두 가지의 복사본을 가지게 된다. cvs rm 명령을 이용하여 이전의 위치에서 파일을 삭제한 후 검사한다. 이 명령은 레포지트리의 Attic디렉터리로 파일을 이동한다. cvs tag -d 명령을 이용하여 새로운 파일의 모든 tag를 제거한다. 이것은 이전의 tag된 버전으로 돌아갈 때 실제로 존재하지 않는(spurious)파일이 나오지 않게 만든다. 날짜를 기준으로 한 checkout는 정확하게 작동하지 않지만 모듈이 적절하게 tag되었다면 필요하지 않을 것이다.

6.5 지운파일 복구하기 ¶

[\[edit\]](#)

만일 소스 트리의 최근 버전에서 파일을 제거하였지만 다시 복구할 필요성이 느껴졌다면, 다음 과정을 따르면 복구할 수 있다. 이것은 cvs add \$file; cvs ci \$file 을 이용한 정교한 작업이다.

cvs status \$file 명령을 이용하여 파일의 두 번째 revision을 찾아 revision 번호를 알아낸다. cvs up -p -r \$rev \$file > \$file 명령을 이용하여 파일의 가장 최근 버전을 불러온다. 필요하다면 파일을 수정한다. cvs add \$file; cvs ci \$file 명령을 이용하여 파일을 레포지트리에 다시 추가하고 check in 한다.

6.6 사용자 정의 Tag ¶

[\[edit\]](#)

Vendor branch에 third-party 소프트웨어를 포함하고 있는 레포트리에는, CVS의 기본 \$Id: CVS_2fGuideLine.v 1.14 2006/07/20 01:18:07 kss Exp kss \$나 \$Header: /home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine.v 1.14 2006/07/20 01:18:07 kss Exp kss \$ tag 보다 사용자 정의 tag를 설정하는 것이 더욱 더 유용하다. 실제 프로젝트인 \$Xorg\$, \$XFree86\$, \$FreeBSD\$, \$NetBSD\$, \$OpenBSD\$, and my own \$dotat\$ 예를 들면, 이 방법의 장점은 파일에 버전이 올라감에 따라 혼란되지 않는 local version 정보(사용자 정의 tag와 기본 tag는 다르다)를 사용자 정의 tag를 이용하여 넣을 수 있다. 사용자 정의 tag를 제외한 모든 tag 확장은 무시된다.

이것을 설정하기 위해서는 CVS를 [FreeBSD](#)나 Debian에 포함된 버전처럼 패치할 필요가 있다. ([NetBSD](#)나 [OpenBSD](#) 버전엔 비슷한 기능이 있지만, 다음 절차는 먹히지 않을 것이다.)

1. 사용할 사용자 정의 tag의 이름을 선택한다. 예를 들면 소속 기관 이름, 이 예에서는 "dotat"

을 사용한다.

2. 레포지트리의 CVSROOT를 check out 한다.
3. CVSROOT 디렉터리 안에 options라는 파일을 다음과 같은 내용을 포함하여 만든다.

```
# Local CVS options:
# Add a "dotat" keyword and restrict keyword expansion
#
# $dotat: doc/web/writing/cvs-guidelines.html,v 1.8 2002/05/07 03:52:22 fanf Exp $
```

```
tag=dotat=CVSHeader
tagexpand=idotat
```

4. options 파일을 평소처럼 cvs add options; cvs commit options 명령을 이용하여 추가한다.
5. 이제 사용자 정의 tag를 사용할 수 있다. 그리고 그 이외의 tag는 확정되지 않는다.

tag 옵션은 \$dotat\$이 \$CVSHeader\$ (기본 \$Header\$:

/home/httpd/kldp/wiki/data/text/RCS/ CVS_2fGuideLine,v 1.14 2006/07/20 01:18:07 kss Exp kss \$와 비슷하지만 CVS root는 제외된다.) tag와 동일어임을 의미한다. tagexpand 옵션은 어떤 tag가 확장될 것인지를 지정한다. 만일 이 인자가 i로 시작한다면 콤마로 분리된 키워드만 확장된다. 만일 이 인자가 e로 시작된다면 리스트된 키워드를 제외하고 모든 tag가 확장된다..

[CategorySoftwareTool](#)

[Tags:](#)

- [cvs](#)
- [버전관리](#)

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

  

Last modified: 2006-07-20 10:18:07, Processing time: 0.0181 sec